

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет безопасности информационных технологий

Направление подготовки: 10.03.01 Информационная безопасность

Образовательная программа: "Информационная безопасность / Information security"

Дисциплина:

«Информационная безопасность баз данных»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

«Реализация сервиса для взаимодействия с разработанной базой данных»

Выполнил студент:

N3346 / ИББД_ТЗИ_N3 1.4

Суханкулиев Мухаммет / 

ФИО

Подпись

Проверил:

Салихов Максим Русланович / _____

ФИО

Подпись

Отметка о выполнении (один из вариантов:
отлично, хорошо, удовлетворительно, зачтено)

Дата

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

Введение	3
1 Обоснование выбора среды разработки и фреймворков	4
2 Взаимодействие сервиса с базой данных	5
2.1 Настройка ORM и маппинг таблиц.....	5
2.1 Настройка ORM и маппинг таблиц.....	7
3 Описание и анализ безопасности сервиса	9
3.1 Назначение, функции и структура	9
3.2 Листинг кода пользовательского интерфейса и окружения.....	9
3.3 Анализ реализованных методов защиты.....	15
Заключение.....	16
Список использованных источников.....	17

ВВЕДЕНИЕ

Цель работы – получение навыков использования серверных языков программирования, фреймворков по работе с БД, ORM-систем.

Задание:

– На основе БД, созданной в предыдущих лабораторных работах, необходимо создать сервис, который взаимодействует с разработанной БД. В качестве сервиса среди прочего могут быть разработаны:

- веб-интерфейс;
- desktop приложение;
- API;
- сервисы предобработки данных, которые заполняют базу данных (обработка websocket соединений, парсинг страниц, json и др)
- сервисы для защиты данных при их обработке, записи, считывании из/в БД (шифрование, обезличивание, сетевая защита и др.)
- (список не исчерпывает все возможные сервисы)

Дайте краткое обоснование выбранной среде разработки, языку и фреймворку, который вы используете для взаимодействия с базой данных.

– В рамках ЛР опишите каким образом вы взаимодействуете с базой данных. С помощью каких фреймворков или языков. Продемонстрируйте несколько примеров по выборке, вставке, удалении данных из вашей БД с помощью выбранного вами фреймворка или языка программирования. Проконтролируйте корректность работы систем логирования и разграничения доступа, которые реализованы в 1-3 ЛР.

– Для реализованного сервиса укажите кратко его основные функции, назначение, кратко опишите структуру. Дополнительно опишите и оцените методы защиты данных, реализованные вами и имеющиеся в наличии в разработанном сервисе. Выделите аналоги, недостатки и преимущества решений, которые вы реализовали.

1 ОБОСНОВАНИЕ ВЫБОРА СРЕДЫ РАЗРАБОТКИ И ФРЕЙМВОРКОВ

Для реализации сервиса был выбран следующий стек технологий:

- Язык программирования: Python. Выбор обусловлен его простотой, большим количеством библиотек для веб-разработки и работы с данными, а также активным сообществом. Python позволяет быстро прототипировать и создавать надежные серверные приложения.
- Веб-фреймворк: Flask. Являясь микро-фреймворком, Flask предоставляет легковесность и гибкость. Он идеально подходит для создания REST API, не навязывая строгую структуру проекта и позволяя разработчику полностью контролировать компоненты приложения.
- ORM-система: SQLAlchemy. Это мощная и гибкая ORM-библиотека для Python, позволяющая работать с реляционными базами данных, используя объектно-ориентированный подход. Использование SQLAlchemy абстрагирует разработчика от написания "сырых" SQL-запросов, что является ключевой мерой защиты от атак класса SQL-инъекций.
- Драйвер для PostgreSQL: psycopg2-binary. Является наиболее популярным и стабильным драйвером для подключения Python-приложений к СУБД PostgreSQL, обеспечивая надежное и производительное соединение.
- Frontend: HTML, CSS, JavaScript (vanilla). Для создания пользовательского интерфейса не использовались тяжеловесные frontend-фреймворки. Применение нативного JavaScript позволяет наглядно продемонстрировать логику взаимодействия с API (отправка запросов, обработка JWT, рендеринг данных).

Данный стек технологий является современным, хорошо документированным и позволяет эффективно решать поставленные задачи, уделяя особое внимание аспектам информационной безопасности.

Для отслеживания всех изменений в данных (операций INSERT, UPDATE, DELETE) будет создана система аудита на основе триггеров.

2 ВЗАИМОДЕЙСТВИЕ СЕРВИСА С БАЗОЙ ДАННЫХ

2.1 Настройка ORM и маппинг таблиц

Взаимодействие с базой данных `scooter_rental_db` осуществляется через SQLAlchemy. Первым шагом является маппинг — описание структуры таблиц БД в виде Python-классов. Это позволяет работать с записями в базе данных как с объектами Python, что упрощает разработку и повышает безопасность.

Листинг 1 – `models.py`

```
from sqlalchemy import (Column, Integer, String, Text, DateTime, Numeric,
                        ForeignKey, CHAR, create_engine, func, LargeBinary)
from sqlalchemy.orm import declarative_base, sessionmaker, relationship
from sqlalchemy.exc import OperationalError

Base = declarative_base()

# --- Описание ORM-моделей для таблиц БД ---

class Zone(Base):
    __tablename__ = 'zone'
    zoneid = Column('ZoneID', Integer, primary_key=True)
    zonename = Column('ZoneName', String(100), nullable=False)
    boundaries = Column('Boundaries', Text)

class Tariff(Base):
    __tablename__ = 'tariff'
    tariffid = Column('TariffID', Integer, primary_key=True)
    tariffname = Column('TariffName', String(50), nullable=False)
    priceperminute = Column('PricePerMinute', Numeric(5, 2), nullable=False)
    startprice = Column('StartPrice', Numeric(5, 2), nullable=False,
                        server_default='0.00')

class User(Base):
    __tablename__ = '"User"'
    userid = Column('UserID', Integer, primary_key=True)
    fio = Column('FIO', String(255), nullable=False)
    phonenumber = Column('PhoneNumber', String(20), unique=True,
                        nullable=False)
    email = Column('Email', String(100), unique=True, nullable=False)
    registrationdate = Column('RegistrationDate', DateTime, nullable=False,
                             server_default=func.now())
    lastlogin = Column('LastLogin', DateTime)

class Scooter(Base):
    __tablename__ = 'scooter'
    scooterid = Column('ScooterID', Integer, primary_key=True)
    model = Column('Model', String(50), nullable=False)
    chargelevel = Column('ChargeLevel', Integer, nullable=False)
    status = Column('Status', String(20), nullable=False,
                    server_default='available')
    zoneid = Column('ZoneID', Integer, ForeignKey('zone.ZoneID'))
    zone = relationship("Zone")

class Maintenance(Base):
    __tablename__ = 'maintenance'
```

```

maintenanceid = Column('MaintenanceID', Integer, primary_key=True)
scooterid = Column('ScooterID', Integer, ForeignKey('scooter.ScooterID'),
    nullable=False)
maintenancetype = Column('MaintenanceType', String(100), nullable=False)
maintenancedate = Column('MaintenanceDate', DateTime, nullable=False)
description = Column('Description', Text)
scooter = relationship("Scooter")

class Trip(Base):
    __tablename__ = 'trip'
    tripid = Column('TripID', Integer, primary_key=True)
    userid = Column('UserID', Integer, ForeignKey('"User".UserID'),
        nullable=False)
    scooterid = Column('ScooterID', Integer, ForeignKey('scooter.ScooterID'),
        nullable=False)
    tariffid = Column('TariffID', Integer, ForeignKey('tariff.TariffID'),
        nullable=False)
    starttime = Column('StartTime', DateTime, nullable=False)
    endtime = Column('EndTime', DateTime)
    cost = Column('Cost', Numeric(8, 2))
    user = relationship("User")
    scooter = relationship("Scooter")
    tariff = relationship("Tariff")

class Payment(Base):
    __tablename__ = 'payment'
    paymentid = Column('PaymentID', Integer, primary_key=True)
    tripid = Column('TripID', Integer, ForeignKey('trip.TripID'),
        nullable=False)
    amount = Column('Amount', Numeric(8, 2), nullable=False)
    paymentdate = Column('PaymentDate', DateTime, nullable=False,
        server_default=func.now())
    paymentstatus = Column('PaymentStatus', String(20), nullable=False,
        server_default='completed')
    trip = relationship("Trip")

class ActionLog(Base):
    __tablename__ = 'action_log'
    log_id = Column(Integer, primary_key=True)
    operation_type = Column(CHAR(1), nullable=False)
    operation_time = Column(DateTime, nullable=False,
        server_default=func.now())
    db_user = Column(Text, nullable=False, server_default=func.current_user())
    table_name = Column(Text, nullable=False)
    old_data = Column(Text)
    new_data = Column(Text)

class UserApiKey(Base):
    __tablename__ = 'user_api_keys'
    key_id = Column(Integer, primary_key=True)
    user_id = Column(Integer, ForeignKey('"User".UserID'), nullable=False)
    api_key = Column(LargeBinary, nullable=False)
    created_at = Column(DateTime, nullable=False, server_default=func.now())

# --- Функция для создания сессии подключения к БД ---

def get_session(role: str, password: str):
    """
    Создает и возвращает сессию SQLAlchemy для указанной роли PostgreSQL.
    Это ключевой механизм безопасности, использующий ролевую модель СУБД.

```

```

:param role: Имя роли (пользователя) в PostgreSQL.
:param password: Пароль для этой роли.
:return: Объект сессии SQLAlchemy или None в случае ошибки подключения.
"""
db_url = f"postgresql+psycopg2://{role}:{password}@localhost:7777/scooter_rental_db"
try:
    engine = create_engine(db_url)

    connection = engine.connect()
    connection.close()

    SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
    return SessionLocal()
except OperationalError as e:
    print(f"ОШИБКА: Не удалось подключиться к базе данных для пользователя '{role}': {e}")
    return None

```

2.1 Настройка ORM и маппинг таблиц

Работа сервиса демонстрирует, что механизмы защиты, реализованные на уровне СУБД в ЛРЗ (разграничение доступа и логирование), функционируют корректно при взаимодействии через API.

Пользователь вводит свои учетные данные в веб-интерфейсе. JavaScript отправляет их на эндпоинт /login. Сервер проверяет данные и в случае успеха возвращает JWT-токен, который сохраняется в браузере.

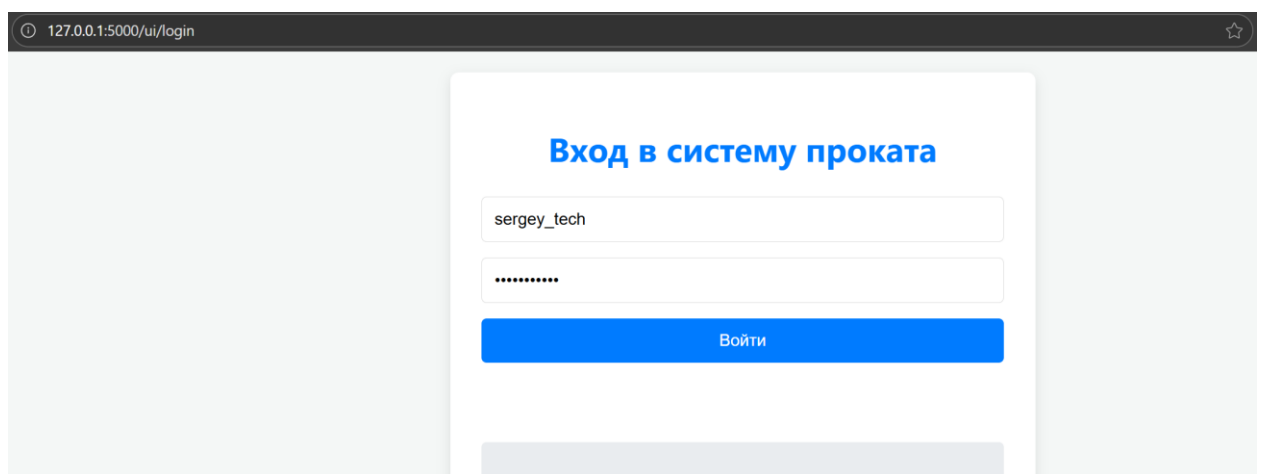


Рисунок 1 – Успешный вход в систему на странице login.html

- После входа пользователь-клиент перенаправляется на свою панель. При нажатии на кнопку "Загрузить доступные самокаты" JavaScript отправляет GET-запрос на эндпоинт /api/client/scooters, прикрепив JWT-токен в заголовок Authorization. Бэкенд:

- Проверяет валидность токена.

- Извлекает из токена роль "client".
 - Подключается к СУБД от имени пользователя с ограниченными правами (anna_vasilieva_client).
 - Выполняет запрос к представлению available_scooters, на которое у данной роли есть права SELECT.
 - Возвращает данные, которые отображаются в виде таблицы.
- Это подтверждает корректную работу системы разграничения доступа.

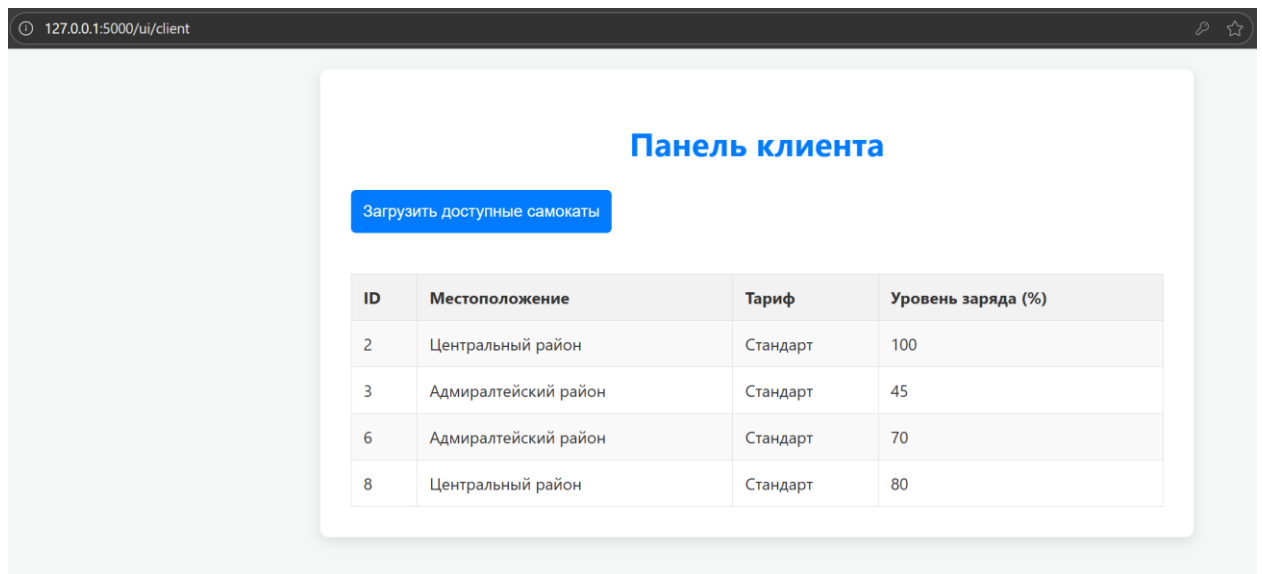


Рисунок 2 – Панель клиента (client_dashboard.html) после успешной загрузки данных

Хотя в интерфейсе не реализованы функции изменения данных, корректная работа триггеров логирования была проверена на этапе разработки. Любая операция INSERT, UPDATE или DELETE, выполненная через сессию SQLAlchemy, корректно фиксируется в таблице action_log. Например, при добавлении новой записи о техобслуживании в лог записывается информация о типе операции ('I'), времени, пользователе (db_user), имени таблицы (maintenance) и JSON-представление новых данных.

3 ОПИСАНИЕ И АНАЛИЗ БЕЗОПАСНОСТИ СЕРВИСА

3.1 Назначение, функции и структура

Назначение: Сервис предоставляет защищенный доступ к данным информационной системы «Прокат электросамокатов» для двух типов пользователей: клиентов и технических специалистов.

Функции:

Аутентификация: Безопасный вход пользователей через API с использованием JWT.

Предоставление данных для клиентов: Просмотр доступных для аренды самокатов.

Предоставление данных для техников: Просмотр журнала технического обслуживания.

Пользовательский интерфейс: Интерактивные веб-страницы для каждой роли.

Структура проекта:

Проект имеет четкую структуру, разделяющую логику бэкенда, статические файлы и шаблоны интерфейса:

```
scooter_api/
├── static/                # CSS-стили
│   └── style.css
├── templates/            # HTML-шаблоны
│   ├── login.html
│   ├── client_dashboard.html
│   └── technician_dashboard.html
├── app.py                # Основной код Flask-приложения (API и UI роутинг)
├── models.py             # ORM-модели SQLAlchemy
├── requirements.txt       # Зависимости проекта
└── run.py                # Скрипт для запуска сервера
```

3.2 Листинг кода пользовательского интерфейса и окружения

Листинг 2 – style.css

```
:root {
  --primary-color: #007bff;
  --background-color: #f4f7f6;
  --container-bg-color: #ffffff;
  --text-color: #333;
  --border-color: #ddd;
  --error-color: #d9534f;
}

body {
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto,
    "Helvetica Neue", Arial, sans-serif;
  background-color: var(--background-color);
  color: var(--text-color);
  margin: 0;
  padding: 20px;
  display: flex;
```

```

        justify-content: center;
        align-items: flex-start;
        min-height: 100vh;
    }

    .container {
        width: 100%;
        max-width: 800px;
        background-color: var(--container-bg-color);
        padding: 30px;
        border-radius: 8px;
        box-shadow: 0 4px 12px rgba(0,0,0,0.1);
    }

    /* Контейнер для логина поуже */
    .login-container {
        max-width: 500px;
    }

    h1 {
        color: var(--primary-color);
        text-align: center;
        margin-bottom: 25px;
    }

    form {
        display: flex;
        flex-direction: column;
        gap: 15px;
    }

    input, button {
        padding: 12px;
        border-radius: 5px;
        border: 1px solid var(--border-color);
        font-size: 16px;
    }

    input:focus {
        outline: none;
        border-color: var(--primary-color);
        box-shadow: 0 0 0 2px rgba(0,123,255,0.25);
    }

    button {
        background-color: var(--primary-color);
        color: white;
        cursor: pointer;
        border: none;
        transition: background-color 0.2s;
        margin-bottom: 20px;
    }

    button:hover {
        background-color: #0056b3;
    }

    table {
        border-collapse: collapse;
        width: 100%;
        margin-top: 20px;
    }

```

```

th, td {
    border: 1px solid var(--border-color);
    padding: 12px;
    text-align: left;
}

th {
    background-color: #f2f2f2;
}

tr:nth-child(even) {
    background-color: #f9f9f9;
}

#error, #result p {
    text-align: center;
    font-weight: bold;
}

#error {
    color: var(--error-color);
    min-height: 20px;
}

#user-guide {
    margin-top: 20px;
    padding: 15px;
    background-color: #e9ecef;
    border-radius: 5px;
    font-size: 14px;
}

code {
    background-color: #d1d5db;
    padding: 2px 5px;
    border-radius: 3px;
}

```

Листинг 3 – login.html

```

<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Вход в систему</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
    <div class="container login-container">
        <h1>Вход в систему проката</h1>
        <form id="login-form">
            <input type="text" id="username" placeholder="Имя пользователя"
            required>
            <input type="password" id="password" placeholder="Пароль"
            required>
            <button type="submit">Войти</button>
        </form>
        <p id="error"></p>
    </div>
</body>
</html>

```

```

        <div id="user-guide">
            <h4>Тестовые пользователи:</h4>
            <ul>
                <li>Клиент:                <code>anna_client</code>                /
            <code>clientpass123</code></li>
                <li>Техник:                <code>sergey_tech</code>                /
            <code>techpass456</code></li>
            </ul>
        </div>
    </div>
    <script>
        const form = document.getElementById('login-form');
        const errorP = document.getElementById('error');
        form.addEventListener('submit', async (event) => {
            event.preventDefault();
            errorP.textContent = '';
            const username = document.getElementById('username').value;
            const password = document.getElementById('password').value;
            try {
                const response = await fetch('/login', {
                    method: 'POST',
                    headers: {'Authorization': 'Basic ' + btoa(username + ':' + password)}
                });
                const data = await response.json();
                if (response.ok) {
                    localStorage.setItem('jwt_token', data.token);
                    if (username.includes('client')) {
                        window.location.href = '/ui/client';
                    } else if (username.includes('tech')) {
                        window.location.href = '/ui/technician';
                    }
                } else {
                    errorP.textContent = 'Ошибка: ' + data.message;
                }
            } catch (error) {
                errorP.textContent = 'Сетевая ошибка или сервер не отвечает.';
            }
        });
    </script>
</body>
</html>

```

Листинг 4 – client_dashboard.html

```

<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Панель клиента</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
    <div class="container">
        <h1>Панель клиента</h1>
        <button id="load-data">Загрузить доступные самокаты</button>
        <div id="result"></div>
    </div>
    <script>
        const loadButton = document.getElementById('load-data');

```

```

const resultDiv = document.getElementById('result');
const token = localStorage.getItem('jwt_token');

if (!token) {
    window.location.href = '/ui/login';
}

loadButton.addEventListener('click', async () => {
    resultDiv.innerHTML = '<p>Загрузка...</p>';
    try {
        const response = await fetch('/api/client/scooters', {
            method: 'GET',
            headers: { 'Authorization': 'Bearer ' + token }
        });

        if (response.status === 401) {
            window.location.href = '/ui/login';
            return;
        }

        const data = await response.json();

        if (response.ok) {
            let table =
'<table><tr><th>ID</th><th>Местоположение</th><th>Тариф</th><th>Уровень
заряда (%)</th></tr>';
            data.forEach(scooter => {
                table += `<tr>
                    <td>${scooter.scooterid}</td>
                    <td>${scooter['Местоположение']}</td>
                    <td>${scooter['Тариф']}</td>
                    <td>${scooter['Уровень заряда']}</td>
                </tr>`;
            });
            table += '</table>';
            resultDiv.innerHTML = table;
        } else {
            resultDiv.innerHTML = `<p style="color:red;">Ошибка:
${data.message}</p>`;
        }
    } catch (error) {
        resultDiv.innerHTML = `<p style="color:red;">Сетевая
ошибка.</p>`;
    }
});
</script>
</body>
</html>

```

Листинг 5 – technician_dashboard.html

```

<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Панель техника</title>
    <link rel="stylesheet" href="{ { url_for('static', filename='style.css')
    } } ">
</head>
<body>
    <div class="container">

```

```

    <h1>Панель техника</h1>
    <button id="load-data">Загрузить журнал обслуживания</button>
    <div id="result"></div>
  </div>
  <script>
    const loadButton = document.getElementById('load-data');
    const resultDiv = document.getElementById('result');
    const token = localStorage.getItem('jwt_token');

    if (!token) {
      window.location.href = '/ui/login';
    }

    loadButton.addEventListener('click', async () => {
      resultDiv.innerHTML = '<p>Загрузка...</p>';
      try {
        const response = await fetch('/api/tech/maintenance_log', {
          method: 'GET',
          headers: { 'Authorization': 'Bearer ' + token }
        });

        if (response.status === 401) {
          window.location.href = '/ui/login';
          return;
        }

        const data = await response.json();

        if (response.ok) {
          let table = '<table><tr><th>ID
самоката</th><th>Модель</th><th>Тип
работ</th><th>Описание</th><th>Дата</th></tr>';
          data.forEach(log => {
            table += `<tr>
              <td>${log.scooterid}</td>
              <td>${log.model}</td>
              <td>${log['Тип работ']}</td>
              <td>${log['Описание работ']}</td>
              <td>${new
обслуживания'}).toLocaleDateString('ru-RU')}</td>
            </tr>`;
          });
          table += '</table>';
          resultDiv.innerHTML = table;
        } else {
          resultDiv.innerHTML = `<p style="color:red;">Ошибка:
${data.message}</p>`;
        }
      } catch (error) {
        resultDiv.innerHTML = `<p style="color:red;">Сетевая
ошибка.</p>`;
      }
    });
  </script>
</body>
</html>

```

3.3 Анализ реализованных методов защиты

Преимущества и реализованные методы защиты:

- Принцип минимальных привилегий: Реализован на уровне СУБД. API-сервис подключается к базе данных от имени ролей с минимально необходимыми правами, что ограничивает ущерб в случае компрометации сервера.
- Защита от SQL-инъекций: Использование SQLAlchemy ORM полностью исключает возможность внедрения вредоносного SQL-кода через параметры запросов.
- Безопасное хранение паролей: Пароли пользователей API не хранятся в открытом виде, а используется их криптографический хэш (bcrypt через werkzeug).
- Аутентификация на основе токенов (JWT): Использование JWT позволяет избежать передачи логина и пароля при каждом запросе, снижая риск их перехвата. Токены имеют ограниченный срок действия (1 час).
- Централизованный аудит: Все изменения данных, инициированные через API, корректно логируются системой аудита на уровне СУБД, созданной в ЛРЗ.
- Разделение ответственностей: Архитектура с разделением на Frontend и Backend API является более безопасной, так как позволяет четко разграничить логику представления и логику доступа к данным.

Недостатки и пути их устранения:

- Передача данных по HTTP: В текущей реализации весь трафик, включая JWT-токен, передается по незащищенному протоколу. Решение: В производственной среде обязательно использование HTTPS (TLS/SSL) для шифрования всего трафика. Это можно настроить с помощью обратного прокси-сервера (например, Nginx).
- Отсутствие механизма отзыва токенов: Если JWT-токен будет скомпрометирован до истечения его срока действия, его нельзя отозвать. Решение: Для критически важных систем можно реализовать "черный список" отозванных токенов (например, в Redis), который будет проверяться при каждом запросе.
- Секретные ключи в коде: Секретный ключ для подписи JWT и учетные данные API жестко закодированы в приложении. Решение: Вынести все секретные данные в переменные окружения или использовать системы управления секретами (например, HashiCorp Vault).

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была успешно достигнута поставленная цель. Разработан многокомпонентный сервис, включающий REST API и пользовательский веб-интерфейс, который безопасно взаимодействует с базой данных PostgreSQL.

Были получены и закреплены практические навыки работы с фреймворком Flask, ORM SQLAlchemy, реализованы современные механизмы аутентификации на основе JWT. Продемонстрирована эффективность многоуровневой защиты, где механизмы безопасности на уровне СУБД (RBAC, аудит) дополняются мерами защиты на уровне приложения (хэширование паролей, проверка токенов).

Разработанный сервис является надежной основой для дальнейшего развития и может быть использован в качестве практической части курсовой работы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы технологий баз данных: учебное пособие / Б. А. Новиков, Е. А. Горшкова, Н. Г. Графеева; под ред. Е. В. Рогова. – 2-е изд. – М.: ДМК Пресс, 2020. – 582 с. – URL: https://drive.google.com/file/d/1TjYbunEjxsbovBiHeYOzBuZrOFonlIRk/view?usp=drive_link
2. Базы данных: Учебник для высших учебных заведений / Под ред. проф. А. Д. Хомоненко. – 6-е изд., доп. - СПб.: КОРОНА-Век, 2009. – 736 с. – URL: https://drive.google.com/file/d/1zIOuO6vdQvb_aVUHGAiucK5MPciUswGf/view?usp=drive_link
3. Psycopg – PostgreSQL database adapter for Python: официальная документация [Электронный ресурс] / The Psycopg Team. – URL: <https://www.postgresql.org/docs/> (дата обращения: 10.10.2025).
4. Официальная документация Flask [Электронный ресурс] – URL: <https://flask.palletsprojects.com/> (дата обращения: 10.10.2025).
5. Официальная документация SQLAlchemy [Электронный ресурс] – URL: <https://www.sqlalchemy.org/> (дата обращения: 10.10.2025).